# Pythonプロジェクトにおける利用証拠付きSBOMの初動トリアージ支援手法の提案と評価

髙士 哲生[1,a)]　依田 みなみ[1,b)]　松野 裕[1,c)]

**概要**：SBOM ベースの脆弱性検査ツールは，パッケージ名とバージョンの照合により既知の脆弱性をアラートとして報告するが，コード内で利用されているかまでは示さない．静的解析結果を SBOM に付加する試みや SSVC などの優先度付け枠組みはあるが，実行時の観測を証拠として保存し，分類に活用する仕組みは確立されていない．このため，実行されないコードへの対応や，実行時に読み込まれるモジュールの検出漏れが生じる．そこで本研究では，import 文から得た静的証拠と実行時観測を SBOM に記録し，アドバイザリと組み合わせて対応区分を決める手法を提案する．利用証拠を保存したうえで対応区分を決めるため，分類方針を変更しても再解析なしに再分類できる．Python プロジェクトの評価では，静的証拠と実行時観測にはそれぞれ一方のみで確認できるパッケージがあり，併用の必要性を確認した．また，検査ツールの 67 件は，利用証拠に基づき優先対応 43 件，後回し 23 件，人の判断を要する 1 件に分類できた．さらに，同じ脆弱なコンポーネントを含む場合でも，アプリケーションごとの利用状況で対応区分に差が生じた．この差は CVSS ベースの重大度や EPSS だけでは区別できない．以上より，利用証拠付き SBOM はアプリケーションごとの利用状況と分類根拠を保持し，脆弱性アラートの初動トリアージを支援する．

**キーワード**：SBOM，脆弱性トリアージ，静的解析，利用証拠，実行時観測

# Design and Evaluation of an Initial Vulnerability Triage Support Method Using Usage-Evidence-Annotated SBOMs for Python Projects

TETSUO TAKASHI[1,a)]　MINAMI YODA[1,b)]　YUTAKA MATSUNO[1,c)]

***Abstract:*** SBOM-based vulnerability scanners report known vulnerabilities as alerts by matching package names and versions, but not whether packages are actually used in the code. Existing efforts append static analysis to SBOMs, and frameworks such as SSVC prioritize alerts; neither stores runtime observations as classification evidence. Consequently, remediation may target never-executed code while missing runtime-loaded modules. We propose recording import-derived static evidence and runtime observations in an SBOM and classifying each alert with advisories. Because evidence is stored before classification, policy changes require no re-analysis. In Python projects, static evidence and runtime observations each captured packages the other missed, confirming both are needed. Usage evidence classified a scanner's 67 alerts into 43 priority, 23 deferral, and one human-judgment case. Even for the same vulnerable component, classes differed by per-application usage, which CVSS-based severity and EPSS alone cannot distinguish. Usage-evidence-annotated SBOMs thus retain per-application usage and rationales, supporting initial triage.

***Keywords:*** SBOM, vulnerability triage, static analysis, usage evidence, runtime observation

## 1. はじめに

Software Bill of Materials（SBOM）は，ソフトウェアを構成するパッケージ名やバージョンなどの一覧である [1].

[1] 日本大学
Nihon University
[a)] cste23057@g.nihon-u.ac.jp
[b)] yoda.minami@nihon-u.ac.jp
[c)] matsuno.yutaka@nihon-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

Software Composition Analysis（SCA）ツールは，SBOM と脆弱性データベースを照合し，既知の脆弱性に該当するコンポーネントをアラートとして報告する．ただし，この照合だけでは，該当パッケージがソースコードから参照されるか，実行時に読み込まれるかは判断できない．Zhou らは 2,414 件の OSS リポジトリを調査し，4 言語 16 リポジトリで報告された 113 件のアラートを人手で確認した．その結果，104 件（92.0%）が偽陽性であり，主な要因は，アプリケーションから実行時に到達しない脆弱なコードの報告であった [2]．この結果は，担当者によるアラートの初動確認において，対象プロジェクトでの利用状況の把握も必要であることを示す．

ソースコードから得た利用状況を SBOM に記録し，アラートの優先順位付けに用いる試みはある [3], [4]．しかし，このような静的解析だけでは，実行時に動的に読み込まれるモジュールまでは確認できない．また，対応優先度を決める枠組みである SSVC [5] は，対象ごとの利用状況を評価項目に定めていない．静的解析の結果と実行時の観測を証拠として保存し，後から分類方針を変えて再分類できるようにするには，証拠の保存と方針の適用とを分離する必要があるが，そのような仕組みは確立されていない．

そこで本研究では，Python プロジェクトを対象に，利用証拠を SBOM の各パッケージに対応付けて保存し，各アラートを優先対応などの対応区分に分類する手法を提案する．利用証拠は，Python ソースから抽出した外部パッケージへのインポートや API 呼び出し候補などの静的証拠と，テストやワークロードの終了時に記録した読み込み済みモジュールの実行時観測からなる．これらの証拠は，パッケージ単位の初動確認に用いる情報である．本手法を次の 3 つのリサーチクエスチョン（RQ）で評価する．

**RQ1** 静的インポート証拠と実行時観測は，互いにどの程度一致するか．また，ソースコード中のインポート名を，標準ライブラリや SBOM 上の配布パッケージの候補へ，どの程度対応付けられるか．

**RQ2** 判定に用いる証拠の種類や分類の規則を変えると，同じ脆弱性アラートへの対応の優先度はどのように変わるか．

**RQ3** 同じ脆弱性情報が複数の OSS に現れるとき，本手法による判定と比べて，利用状況を考慮しない既存指標は，OSS ごとの対応の違いをどこまで表せるか．

評価の結果，静的証拠と実行時観測にはそれぞれ一方のみで確認できるパッケージがあった．また，証拠や分類の規則によって対応の優先度は変化し，同じ脆弱性情報でも対象プロジェクトによって対応の優先度が異なる場合があった．これらの違いは，脆弱性ごとの既存指標だけでは説明できなかった．以上より，本手法は対象ごとの利用状況と分類の根拠を示し，初動トリアージを支援する．

本稿の貢献は以下の 3 点である．(1) 利用証拠を抽出する軽量な静的解析器と CI パイプラインを実装した．(2) 既存の単純方式と複数の分類方針を同じ証拠へ適用し，対応区分が変わる要因を示した．(3) 脆弱性単位の指標だけでは表せない対象ごとの利用状況があることを示した．

## 2. 関連研究

### 2.1 SBOM の生成・利用状況の付加と対応優先度の判断

Benedetti らは，Python 向けの SBOM 生成ツールで検出されるパッケージや依存関係が異なると報告した [6]．これに対し本研究は，ツール間の差ではなく，生成された SBOM に利用証拠を付加し，アラートごとの対応区分を判断する．

SBOM に利用状況を記録する枠組みは既にある．CycloneDX の `component.evidence` は，パッケージを検出したファイルや方法，ソースコード中の出現箇所，呼び出し元を記録できる [7]．SBOM 生成ツール cdxgen の拡張機能 evinse は，ソースコードを解析し，各パッケージが呼び出されうる箇所やサービスの情報を SBOM に付加し [3]，脆弱性検査ツール OWASP dep-scan はこの情報をアラートの優先順位付けに用いる [4]．また，対応優先度を決める枠組みである SSVC [5] は，悪用の有無や影響の大きさなどの判断材料を評価するが，パッケージの利用状況を評価項目に定めていない．本研究は，静的証拠，実行時観測，SBOM の部品へ対応付けられない情報を分けて保存し，静的証拠と実行時観測を区別した対応区分を示す．

### 2.2 コード到達性と静的・動的解析

脆弱なライブラリが依存関係に含まれることだけでなく，その脆弱なコードがアプリケーションから到達可能かを調べる研究がある．Ponta らは，脆弱性の修正箇所とアプリケーション側の呼び出し箇所を対応付け，脆弱なコードの利用を解析した [8]．Foo らは，静的解析と動的解析から得た呼び出し関係を組み合わせ，単独では捉えにくい利用経路を補完した [9]．Mir らは Maven を対象に，脆弱な依存が含まれることと，その依存内の脆弱なコードへの到達を分けて評価し，依存の推移性と解析粒度によって到達の判定結果が変わることを示した [10]．Python 向けの PyCG [11] も呼び出しグラフを生成する．これらの研究が関数や呼び出し関係の解析を中心とするのに対し，本研究は実行時観測も証拠として保存し，パッケージ単位で初動確認の対象を選べるようにする．

## 3. 提案手法

提案手法は，利用証拠を生成して SBOM と詳細証拠ファイルへ保存する処理と，保存済みの証拠をアドバイザリへ結合して対応区分を決める処理からなる．後者で用いる，証拠の参照順序と対応区分への振り分け方を定めた規則を分類方針と呼ぶ．**図 1** に全体像を示す．

© 2026 Information Processing Society of Japan

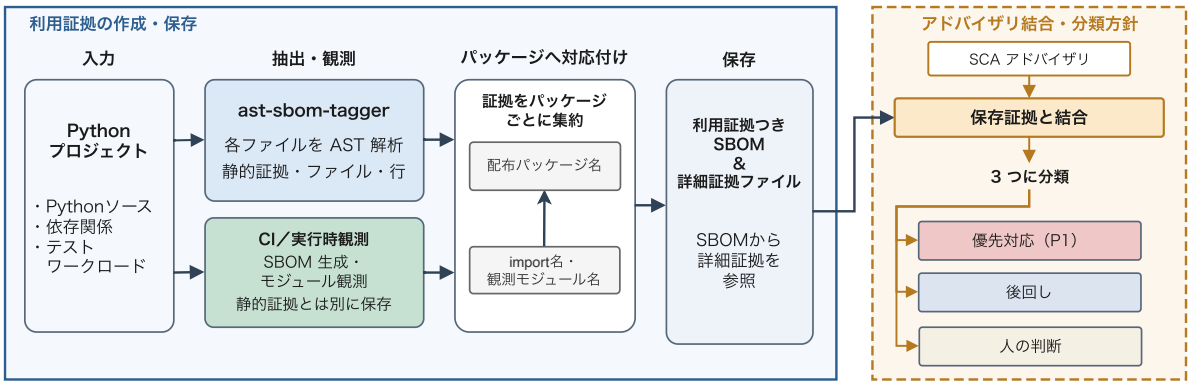



**図 1**　提案手法の処理フロー

図の左側では，環境と SBOM を準備し，静的証拠と実行時観測を SBOM 部品へ対応付けて保存する．右側では，保存した証拠へアドバイザリを結合し，分類方針を適用する．以下では，入出力と前提を示した後，この流れを 6 手順に分けて説明する．

### 3.1　入力，出力，前提条件

本研究では，Python ソースから静的な利用証拠を抽出する解析ツール ast-sbom-tagger と，SBOM 生成，実行時モジュール観測，証拠保存を行う CI パイプライン annotated-sbom-ci を実装し，公開した [12]．本稿では，脆弱性データベースに登録された個々の脆弱性情報をアドバイザリ，その識別子をアドバイザリ ID と呼ぶ．証拠生成には解析対象の Python プロジェクトとテストまたは代表ワークロードを，分類には生成した SBOM，詳細証拠ファイル，アドバイザリを用いる．

本稿では，`import` 文または `from ... import ...` 文に現れる先頭モジュール名をインポート名，SBOM や Python 環境で配布単位を表す名前を配布パッケージ名と呼ぶ．例えば，`import yaml` のインポート名は `yaml` であるが，対応する配布パッケージ名は `PyYAML` である．出力は，パッケージごとの証拠の種類・件数・実行時観測の状態を記録した CycloneDX SBOM と，ファイル名・行番号・実行条件・未解決情報を記録した詳細証拠ファイルである．SBOM から詳細証拠ファイルを `externalReferences` で参照し [7]，SBOM 部品へ対応付けられない証拠も未解決情報として残す．

### 3.2　処理手順

(1) CI パイプラインは，pytest の実行に必要な追加パッケージを導入する前の隔離環境にあるパッケージのメタデータから SBOM を生成するとともに，インポート名と配布パッケージ名の対応表を自動作成する．

(2) ast-sbom-tagger は各 Python ファイルを抽象構文木（Abstract Syntax Tree; AST）へ変換する．外部 import，API 呼び出し候補，設定文字列，依存注入，フレームワークの初期化呼び出し，動的 import 候補を抽出し，ローカルモジュールと標準ライブラリを外部パッケージから除く．条件分岐や `try` ブロック内の import・API 呼び出し候補，関数内の import といった出現文脈も，抽出位置とともに記録する．動的 import は，`importlib.import_module` と `__import__` の第 1 引数の式を静的に解析し，読み込み先のモジュール名を特定できた場合に候補名を記録する．候補名を得られない式は，未解決の読み込み候補として保存する．

(3) 対象ごとに選んだテストまたは代表ワークロードを実行し，終了時に読み込まれていたモジュールを実行記録として保存する．

(4) 対応表と SBOM を用い，抽出した静的証拠と実行記録のモジュールを配布パッケージへ対応付ける．SBOM に記録された各配布パッケージは，対応するモジュールのいずれかが実行記録にあれば `observed`，実行記録はあるが対応するモジュールがなければ `unobserved`，実行記録を得られなければ `unknown` とする．

(5) パッケージごとに証拠の種類，件数，実行時観測の結果をまとめて SBOM へ付加する．個々の静的証拠と抽出位置は詳細証拠に残し，SBOM 部品へ対応付けられない証拠も，テスト用・開発用依存の観測を含め，別に保存する．また，SBOM の依存関係を用い，静的証拠または実行時観測があるパッケージからたどれる依存パッケージを，推移的な利用候補として記録する．

(6) 保存済みの証拠とアドバイザリを結合し，分類方針に従って各アラートの対応区分をそれぞれ決める．

手順 1–5 は通常の CI で利用証拠を準備・保存する処理であり，初動確認が必要になったときに手順 6 へ進む．手順 6 は保存済みの証拠を参照するため，アドバイザリの追加や分類方針の変更にも，ソースコードを再解析せずに対応区分を決め直せる．

© 2026 Information Processing Society of Japan

### 3.3 判定基準と設計理由

本稿では，アラート 1 件ごとに，どのように対応するかを表す 7 種類の区分を対応区分と呼び，利用証拠から対応区分を決める．本評価では，その決め方の一例として，各パッケージについて次の規則を上から順に調べ，最初に該当する規則の対応区分を採用する．

(1) パッケージが実行時のインストール集合になく，テスト用・開発用依存の集合だけに含まれる場合は，テスト・開発用（test/dev）とする．
(2) `pip` や `wheel` など，環境構築用として定めたパッケージは，環境構築とする．
(3) 対象別に定めた代表ワークロードで観測されたパッケージは，優先対応（P1）とする．
(4) 実行時に用いるソースコードから直接 import されたパッケージは，静的確認とする．
(5) SBOM の依存関係をたどって得た推移的な利用候補は，推移確認とする．
(6) テスト実行だけで観測されたパッケージは，テスト・開発用とする．
(7) 依存宣言または設定文字列だけに現れるパッケージは，宣言のみ確認とする．
(8) 上記のいずれにも該当しないパッケージは，人手確認とする．

SBOM 部品と対応付けられない証拠は判定に反映せず，詳細証拠に未解決情報として残す．規則 (1) と (6) はいずれもテスト・開発用とするため，得られる対応区分は 7 種類である．本稿では評価のため，これらを P1，後回しとする 4 区分（テスト・開発用，環境構築，静的確認，推移確認），人による判断が必要な 2 区分（宣言のみ確認，人手確認）の 3 群にまとめる．P1 以外の 2 群を非 P1 と呼ぶが，人による判断が必要な区分は後回し件数に含めない．

## 4. 評価対象と共通条件

本章では，RQ1–RQ3 に先立ち，主評価対象と，複数の RQ に共通するアラートの集計基準，代表ワークロード，およびアドバイザリの固定条件を示す．

### 4.1 主評価対象と集計基準

主評価対象は requests 2.31.0，Flask 3.0.0，Scrapy 2.11.2 の 3 OSS と，著者が開発した ShiftManager 2.0.0 の計 4 つの Python プロジェクトである．主評価の RQ2 と RQ3 では，Python 環境のパッケージ名とバージョンを既知脆弱性と照合する pip-audit [13], [14] の出力を用いる．テスト用・開発用依存を導入した環境に対して pip-audit を実行し，一つの配布パッケージに一つのアドバイザリが報告された状態をアラート 1 件と定義すると，対象別に 29 件，13 件，2 件，23 件の計 67 件である．この 67 件には requests のテスト用・開発用依存に対する 18 件を含み，アラート集合は 2026 年 5 月に保存した pip-audit の出力結果として固定した．

### 4.2 共通ワークロードとデータ固定条件

主評価の代表ワークロードは，requests の送信直前までの GET 要求準備，Flask のテストクライアントによる HTML・JSON 応答の取得，Scrapy の固定 HTML からの抽出，ShiftManager のアプリケーション読み込み・認証・入力モデル生成とした．これらの代表ワークロードは，RQ1 の集合比較と，RQ2 と RQ3 の T2 の判定に共通して用いる．

## 5. RQ1：静的証拠と実行時観測の一致およびインポート名の判定

### 5.1 評価方法と分析対象

**主評価と実行時観測の補助評価**

静的な import 証拠があるパッケージ集合を $S$，テストまたは代表ワークロード中に読み込みを観測したパッケージ集合を $D$ とする．$S$ と $D$ からは，`pip`, `setuptools`, `wheel` など，環境構築用として定めたパッケージを除く．静的に見つけた候補のうち実行時にも確認できた割合 $|S \cap D|/|S|$ と，実行時に確認したパッケージのうち静的にも見つかった割合 $|S \cap D|/|D|$ を求める．

主評価 4 対象では分類に用いた代表ワークロードを $D$ とする．これとは別に，7 対象を実行時観測の補助評価とする．Click，CTFd，SearXNG，wger ではテストを実行し，aiohttp，Kedro，Celery では対象の主要モジュールを一度読み込む簡易実行を行った．重複を除くインポート名のうち，標準ライブラリと判定できたもの，または一つ以上の配布パッケージ候補を得たものの割合をインポート名判定率とする．

**既存手法との補助比較**

静的解析の補助比較では，呼び出しグラフ生成器 PyCG [11] を 16 対象へ適用する．出力されたグラフにおける各ノードの先頭モジュール名を，配布パッケージ名の別名または対応表で一意に定まるインポート名と照合する．この照合結果に基づき，本手法の静的 import 検出結果に対する PyCG のモジュール出現割合を測定する．PyCG は公式アーカイブ版を Python 3.10.18 で実行し，処理資源を制限するため，呼び出し関係の更新回数は 1 回に固定した．静的解析単体の処理時間は，PyCG が呼び出しグラフを出力した 11 対象（計 572,687 行）で計測する．各対象を独立したプロセスで 3 回解析し，3 回の中央値を代表値として集計する．測定環境は Kali GNU/Linux 2026.1（Core i7-13700F，32 GB メモリ）である．ast-sbom-tagger の計測区間は `scan` だけとし，ソースコードの取得，依存の導入，SBOM と対応表の生成，テスト，実行時観測を含めない．

© 2026 Information Processing Society of Japan

表 1　静的候補 $S$ と実行時観測 $D$ の比較

| 対象 | $\lvert S\rvert$ | $\lvert D\rvert$ | $\lvert S\cap D\rvert/\lvert S\rvert$ | $\lvert S\cap D\rvert/\lvert D\rvert$ |
|---|---|---|---|---|
| 主評価の 4 対象 | | | | |
| requests 2.31.0 | 5 | 5 | 1.00 | 1.00 |
| Flask 3.0.0 | 7 | 7 | 1.00 | 1.00 |
| Scrapy 2.11.2 | 26 | 23 | 0.50 | 0.57 |
| ShiftManager 2.0.0 | 7 | 17 | 0.86 | 0.35 |
| 補助評価 7 対象 | | | | |
| Click | 1 | 1 | 1.00 | 1.00 |
| aiohttp | 6 | 9 | 1.00 | 0.67 |
| CTFd | 14 | 59 | 0.86 | 0.20 |
| SearXNG | 13 | 33 | 0.85 | 0.33 |
| wger | 36 | 68 | 0.58 | 0.31 |
| Kedro | 23 | 1 | 0.04 | 1.00 |
| Celery | 6 | 1 | 0.17 | 1.00 |

既存ツール比較には，cdxgen/evinse 12.8.2（Atom 2.5.6 同梱）を用い，主評価 4 対象と，Zhou らの確認済み Python 4 対象（beets，diagrams，cryptography，pendulum）の計 8 対象を用いる．上記 8 対象に対して，cdxgen で利用箇所情報を含む SBOM を生成し，evinse の Atom を用い静的解析でソースコードにおける呼び出しやデータの受け渡しによるフローを記録する．本稿ではこの記録を到達フローと呼ぶ．対応する配布パッケージを特定できたフローにはパッケージ識別子 purl（package URL）が付くため，部品の出現位置と，purl によるフローと SBOM 部品の対応を確認する．

### 5.2　評価結果

#### 主評価と実行時観測の補助評価

表 1 に $S$ と $D$ の比較を示す．主評価 4 対象で静的な import 証拠を持つ対象・配布パッケージは 45 組であり，そのうち 31 組（68.9%）を実行時にも確認した．requests と Flask では静的候補のすべてが実行時にも観測され，Scrapy は 13/26 組，ShiftManager は 6/7 組であった．一方，実行時に確認したパッケージを基準にすると，Scrapy では 13/23 組，ShiftManager では 6/17 組に静的証拠があり，実行時観測で初めて捉えたパッケージもあった．

補助評価では 7 対象すべてについて，評価方法で定めたテストまたは主要モジュールの簡易読み込みを実行した．このうち 5 対象では，静的に見つけたパッケージの半数以上を実行時にも確認した．Kedro と Celery では実行時に観測できたのは各 1 組のみであった．

インポート名判定率は，主評価 4 対象では最低 87%，最高 96%，実行時観測の補助評価 7 対象では最低 76%，最高 98%（中央値 89%）であった．

#### 既存手法との補助比較

PyCG との補助比較では，16 対象のうち 11 対象で PyCG が呼び出しグラフを出力した．この 11 対象の SBOM に記載のある外部パッケージ 204 件のうち，静的 import 証拠があった 98 件の 92.9%（91 件）は PyCG の呼び出しグラフにも現れた．それ以外では本手法のみに現れたものは 7 件，PyCG のみに現れたものは 0 件，どちらにも現れなかったものは 106 件であった．

PyCG が完了しなかった 5 対象では，ansible-core で ImportError，Kedro で SyntaxError，Django，JupyterHub，Werkzeug で PyCG 内部の IndexError が発生したため，比較から外した．

本手法の `scan` を各対象で 3 回実行し，各対象の中央値を合計すると 6.332 秒であった．11 対象の Python ファイルは計 572,687 行であり，1 秒当たり約 90.4 千行に相当する．

上記 8 対象では，それぞれ 2–26 部品に利用箇所情報が付与され，7 対象では SBOM 部品に結び付く purl 付き到達フローが得られた．pendulum では到達フロー自体が 0 件であった．beets の purl 付き到達フロー 214 件のうち 207 件はテストコードと pytest の purl を含み，残る 7 件では，`docs/conf.py` への参照が無関係な外部パッケージ `django-docs` へ誤って対応付けられていた．本手法はインポート名 `docs` の対応先を決めず，未解決とした．

### 5.3　RQ1 への回答

RQ1 への回答として，インポート名判定率は主評価 4 対象で 87–96%，補助評価 7 対象で 76–98%であった．主評価 4 対象では，表 1 に示した静的証拠 45 組中 31 組を実行時にも確認し，対象別の $\lvert S\cap D\rvert/\lvert S\rvert$ は 0.50–1.00，$\lvert S\cap D\rvert/\lvert D\rvert$ は 0.35–1.00 であった．

## 6.　RQ2：判定情報と分類方針による対応区分の差

### 6.1　評価方法と分析対象

#### 主評価

RQ2 では，主評価 67 件の脆弱性アラートを T1a，T1b，T2，import-only の 4 方式で P1 と非 P1 に分ける．対象間で依存の範囲をそろえた感度確認として，requests のテスト用・開発用依存 18 件を除いた 49 件についても T2 の分類を行う．

**T1a**　実行時依存だけを導入した環境で `pip freeze --all` を実行し，その出力を保存する．通常の `name==version` 形式の行からパッケージ名集合を作る．集合にないパッケージと環境構築用パッケージを非 P1，それ以外を P1 とする．

**T1b**　T1a の集合に解析対象自身の配布パッケージ名を明示的に補い，`-e` 形式の行を除外した T1a で対象名が集合から欠ける影響を確認する．

© 2026 Information Processing Society of Japan

**T2**　パッケージの用途，環境構築用パッケージ，静的証拠，実行時観測に 3.3 節の分類方針を適用して対応区分を決め，P1 とそれ以外の非 P1 に分ける．

**import-only**　対象内のすべての Python ファイルにある import 文だけを用い，配布パッケージ名に対応するインポート名が一度でも現れれば P1，現れなければ非 P1 とする．

各方法の P1 件数，非 P1 件数，非 P1 率と，T1a，T1b，import-only の各方式が T2 と異なる区分にした件数を求める．区分が異なったアラートでは，各方式が参照した `pip freeze --all` の出力，静的証拠，実行時観測を確認し，違いを生んだ情報を調べる．

**既存ツールとの補助比較**

RQ1 の補助比較に用いた 8 対象について，cdxgen/evinse が出力した到達フローを OWASP dep-scan 6.3.0 へ入力し，脆弱性の対象パッケージが到達フローに現れたか，到達表示が付いたか，優先して確認する対象に選ばれたかを調べる．同じ SBOM と脆弱性 DB を用い，到達フローの有無だけを変えて dep-scan を実行する．また，対象，配布パッケージ，CVE が共通する組について，dep-scan の結果とこの 8 対象の保存証拠に 3.3 節の分類方針を適用した結果を照合する．

**分類方針の追加評価**

保存した証拠を変えずに対応区分の決め方だけを変える追加評価では，3.3 節の分類方針と同様に，運用組織が採用しうる代替方針の例として 4 方針を定め，主評価とは別の 8 対象（aiohttp，CTFd，Kedro，dbt-core，mitmproxy，Prefect，SearXNG，wger）へ適用する．対象・配布パッケージ・別名統合後アドバイザリ群の組を 1 件と数えた 185 件であり，主評価 67 件とはアドバイザリの計数単位が異なる．この 185 件には，2026 年 6–7 月に取得した，主評価とは別世代の保存記録を用いる．4 方針は，(i) 静的証拠または pytest 観測があるものを優先対応とする，(ii) 種類を問わず利用証拠があれば P1 とする，(iii) 別プロセスで動きうる Web サーバ等の運用部品を専用区分に分ける，(iv) 証拠がないパッケージも P1 とする，である．専用区分を含め，対象自身，静的証拠，実行時観測，環境・Web サーバ用パッケージのいずれかに基づく区分を「今確認」，証拠がない場合の区分を「保留」にまとめ，境界が変わるアラートと利用証拠を調べる．

### 6.2　評価結果

**主評価**

表 2 に主評価 67 件の比較を示す．提案手法 T2 は，代表ワークロードで読み込みを確認した 43 件を P1，23 件を後回し，設定文字列から候補を得た 1 件を，人による判断が必要な「宣言のみ確認」とした．4 対象からテスト用・開発用依存 18 件を除いた 49 件では，非 P1 は 6 件（12.2%）となり，67 件で見られた非 P1 の大半はテスト・開発用の依存に由来したためである．

**表 2**　主評価の 67 件における対応区分の比較

| 方法 | P1 | 非 P1 | 非 P1 率 |
|---|---|---|---|
| T1a: `pip freeze --all`（通常行） | 40 | 27 | 40.3% |
| T1b: T1a ＋対象自身 | 44 | 23 | 34.3% |
| T2: 提案手法の分類方針 | 43 | 24 | 35.8% |
| import-only | 31 | 36 | 53.7% |

対象別では，requests の 29 件中 18 件をテスト用・開発用依存として後回しにした．ShiftManager では 23 件中 5 件を環境構築用パッケージとして後回しにし，1 件を「宣言のみ確認」とした．Flask の 13 件と Scrapy の 2 件はすべて P1 であった．この結果，非 P1 率は requests で 62.1%，ShiftManager で 26.1%，Flask と Scrapy で 0%となった．

T1a と T2 が異なった 5 件のうち 4 件は，`pip freeze --all` の出力形式による差であった．この 4 件は解析対象自身のアラート（requests 自身の 3 件と Flask 自身の 1 件）で，対象自身は同コマンドでは `name==version` 形式ではなく，`-e` から始まるソース URL 形式で出力される．T1a は通常形式の行だけから名前集合を作るため，この 4 件を非 P1 とした．T1b で対象自身の名前を補うと 4 件はいずれも P1 になり，この変化は利用証拠によるものではない．残る 1 件は，利用証拠が判定を変えた例である．ShiftManager の PyMySQL は T1b では P1 であるが，T2 では，接続文字列 `mysql+pymysql://...` から MySQL 接続に PyMySQL を使う指定を確認した一方，import 文も代表ワークロードでの読み込みも確認されなかったため，「宣言のみ確認」とした．import-only と T2 が異なった 14 件のうち 13 件は，import-only では非 P1 であったが，代表ワークロードでの読み込みを確認した T2 では P1 となった．残る 1 件は pytest で，import-only では P1 であったが，テスト用・開発用依存を区別する T2 では非 P1 となった．

**既存ツールとの補助比較**

dep-scan が 8 対象について出力した計 223 件の脆弱性レコードは到達フローの有無で変わらず，入力すると 38 件に到達表示が付き，2 件が優先して確認する対象となったが，入力しない場合はどちらも 0 件であった．dep-scan が優先して確認する対象とした 2 件のうち 1 件は，Scrapy の対象バージョン `2.11.2` には該当しない CVE であるため，照合から除いた．残る 1 件を含め，対象，配布パッケージ，CVE が共通する 55 件を照合すると，優先対象とした 1 件は本手法でも P1 であり，優先対象としなかった 54 件は，本手法では P1 38 件，テスト・開発用 15 件，宣言のみ確認 1 件に分かれた．

**分類方針の追加評価**

保存済みの詳細証拠を変えずに 4 種類の分類方針を適用

© 2026 Information Processing Society of Japan

し直したところ，「今確認」と「保留」の区分が方針間で異なったのは，8 対象 185 件中 8 件（4.3%）であり，残る 177 件ではこの境界がすべての方針で一致した．内訳は CTFd の gunicorn 2 件，mitmproxy の protobuf 1 件，Prefect の urllib3 5 件である．8 件には静的証拠も実行時観測もなく，証拠のないアラートを「今確認」とするか「保留」とするかが方針間で異なった．

### 6.3 RQ2 への回答

RQ2 への回答として，実行時依存一覧の出力形式，対象自身の名前の補完，実行時観測，依存用途の区別によって，同じ脆弱性アラートの対応区分が変わった．T2 と異なる区分になったのは T1a で 5 件，T1b で 1 件，import-only で 14 件であり，依存範囲をそろえた 49 件では T2 の非 P1 は 6 件（12.2%）となった．証拠がないアラートでは，分類方針によって「今確認」と「保留」の境界が変化した．

## 7. RQ3：既存指標による OSS ごとの対応区分の区別可能性

### 7.1 評価方法と分析対象

OSS 間で同じ脆弱性をまとめるため，CVE ID がある場合は CVE ID を，ない場合はアドバイザリ ID を脆弱性識別子とする．主評価 67 件について，同じ識別子が P1 と非 P1 の両方に現れる数を求め，対象と対応区分の組合せを確認する．

Exploit Prediction Scoring System（EPSS）は今後 30 日間に悪用活動が観測される確率を示す [15]．EPSS は 2026 年 6 月 10 日時点の値を用いる．EPSS には対応の優先度を分ける確立した閾値がないため，T2 で後回しとした件数と同じ 23 件を EPSS 値が低い順に後回し候補と仮定し，この中に T2 の P1 が何件含まれるかを調べる．重大度には，保存した OSV アドバイザリの `database_specific.severity` に記録された CVSS ベースの区分を用い，重大度 HIGH 以上を優先対応の目安とし，それ未満のアラートを同じく後回し候補として，T2 と比較する．EPSS を取得できたのは 65 件，重大度を取得できたのは 63 件である．T2 で後回しとした 23 件はいずれも EPSS を取得できた．EPSS や脆弱性単位の重大度は，同じ脆弱性が複数の OSS に現れても値が変わらない．

### 7.2 評価結果

主評価 67 件では，65 件に CVE ID を，CVE ID のない 2 件に GHSA のアドバイザリ ID を用いた．CVE ID または GHSA ID が同じアラートを一つとして数えると 54 個であり，このうち 12 個（22.2%）は，requests では非 P1，Flask では P1 となった．

EPSS を取得できた 65 件から，T2 の後回し件数に合わせて EPSS が低い 23 件を選ぶと，そのうち 13 件は T2 では P1 であった．重大度を取得できた 63 件では，HIGH 未満の 42 件中 25 件が T2 において P1 であった．したがって，EPSS または重大度だけによる選択は，対象ごとの利用証拠を用いてアラートを分類した T2 と一致しなかった．

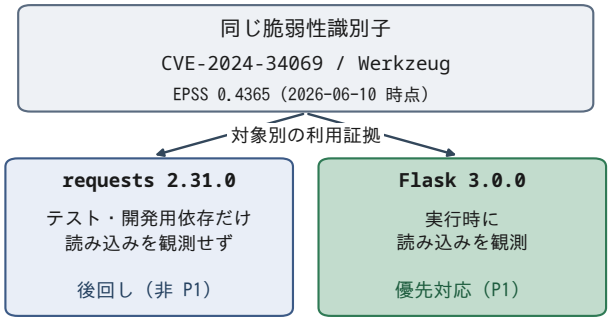



**図 2** 同一脆弱性識別子に対する OSS 別の利用証拠と対応区分

**図 2** に，CVE-2024-34069 の例を示す．EPSS は両 OSS で同じ 0.4365 である．requests では Werkzeug がテスト用・開発用依存だけにあり後回しとなり，Flask では実行時に読み込みが観測されたため P1 となった．

対応区分が異なった 12 個を配布パッケージ別に確認すると，Werkzeug 7 個，Jinja2 4 個，Flask 1 個であり，すべて requests と Flask の比較から生じた．

### 7.3 RQ3 への回答

RQ3 への回答として，主評価の 54 個の脆弱性識別子のうち 12 個は requests と Flask で対応区分が異なった．また，EPSS が低い 23 件には T2 の P1 が 13 件，重大度が HIGH 未満の 42 件には T2 の P1 が 25 件含まれた．したがって，本標本では，これらの脆弱性単位の指標だけでは対象ごとの対応区分の違いを表せなかった．

## 8. 考察

### 8.1 利用証拠と単純方式との差

利用証拠の役割は，非 P1 件数を増やすことではなく，各アラートをその区分にした根拠と，追加確認が必要な点を示すことにある．実際，T1b と T2 の非 P1 件数は近かったが，T2 は設定文字列だけに現れた PyMySQL 1 件を，人による判断が必要な区分とした．T1a の非 P1 には `pip freeze --all` の出力形式による欠落が，import-only の非 P1 には実行時観測の欠落が影響し，非 P1 の件数は評価に含める依存の範囲にも左右された．

ShiftManager の実行時観測のみ 11 件と Scrapy の静的証拠のみ 13 件から，両証拠が異なるパッケージを補完したことを確認した．対象，配布パッケージ，CVE が共通する 55 件中，dep-scan が優先対象外とした 54 件にも，本手法は実行時の読み込みと依存用途に基づき 3 区分を示した．

© 2026 Information Processing Society of Japan

### 8.2 分類方針の変更と保存証拠の役割

証拠の生成と分類方針を分けることで，ソースコードを再解析せず，組織の判断に応じて「今確認」と「保留」の境界を決め直せる．比較した 4 方針で境界が異なったのは，静的証拠と実行時観測を持たない 8 件のみであった．証拠がないことは未使用を意味しないため，その状態を保存しておけば，分類方針を変えたときに区分が変わるアラートを確認できる．

### 8.3 対象固有の利用状況と既存指標の補完関係

脆弱性単位の重大度や EPSS だけでは，解析対象ごとの利用状況の違いは表せない．54 個の脆弱性識別子のうち 12 個は requests と Flask で区分が異なり，EPSS が低い 23 件にも T2 の P1 が 13 件含まれた．EPSS は悪用の確率，重大度は脆弱性の深刻さ，利用証拠は対象固有の利用状況を表し，これらは異なる判断材料になる．実際の悪用や修正判断との一致は今後の評価課題である．運用では全アラートを残し，利用証拠による対応区分と重大度，EPSS を併記して，組織の分類方針に従って確認順を決定する．

### 8.4 運用方法と適用の限界

第一に，主評価は 4 対象であり，区分の差も requests と Flask の 1 組から得られたため，出現頻度の一般化には対象の追加が必要である．第二に，実行時観測はテストまたはワークロードを実行した Python プロセスだけを対象とするため，終了時のモジュール記録に残らない動的プラグインや C 拡張，別プロセスで読み込まれるモジュールは観測から漏れることがある．第三に，本手法はパッケージ単位の初動確認を支援するものであり，脆弱な関数への到達や悪用可能性までは判定しない．第四に，ソースコード，SBOM，ワークロードが変わった場合に，どの証拠を再生成する必要があるかは評価していない．また，分類方針の追加評価は詳細証拠がある 8 対象に限られる．

## 9. おわりに

本研究は，静的インポート証拠と実行時観測を SBOM のパッケージへ対応付け，証拠の生成・保存と分類方針を分離する手法を提案し，両証拠の一致と分類方針による区分の差の両面から評価した．評価では，静的証拠の多くを実行時にも確認できた一方，脆弱性アラート 67 件の対応区分は判定情報と分類方針で変わり，脆弱性識別子 54 個中 12 個は対象間で区分が異なった．

以上から，本評価の範囲では，利用証拠付き SBOM は，重大度や EPSS だけでは表せない対象ごとの利用状況と分類根拠を保持する手段として位置付けられる．今後は，対象とワークロードを追加して両証拠の補完の度合いを確かめ，区分が対象間で異なる頻度を調べる．次に，どちらの証拠も持たないアラートを減らすため，観測範囲を複数プロセス，動的プラグインや C 拡張へ広げる．また，詳細証拠に保存した API 呼び出し候補をアドバイザリの脆弱な関数の情報と照合し，関数単位の判定を試みる．

## 謝辞

本研究は JSPS 科研費 JP25K21182 の助成を受けたものです．

### 参考文献

[1] NTIA, : The Minimum Elements for a Software Bill of Materials (SBOM), U.S. Department of Commerce (2021), `https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom`.

[2] Zhou, L., Dacier, M. and Konstantinou, C.: A Reality Check on SBOM-based Vulnerability Management: An Empirical Study and A Path Forward (2026).

[3] cdxgen Project, : CycloneDX Generator, `https://github.com/cdxgen/cdxgen`: accessed 2026-08-02.

[4] OWASP Foundation, : OWASP dep-scan, `https://owasp.org/www-project-dep-scan/`: accessed 2026-08-09.

[5] Spring, J., Householder, A., Hatleback, E., Manion, A., Oliver, M., Sarvepalli, V., Tyzenhaus, L. and Yarbrough, C.: Prioritizing Vulnerability Response: A Stakeholder-Specific Vulnerability Categorization (2021).

[6] Benedetti, G., Cofano, S., Brighente, A. and Conti, M.: The Impact of SBOM Generators on Vulnerability Assessment in Python: A Comparison and a Novel Approach (2025).

[7] OWASP Foundation, : CycloneDX Bill of Materials Standard, `https://cyclonedx.org/specification/overview/`: accessed 2026-08-02.

[8] Ponta, S. E., Plate, H. and Sabetta, A.: Beyond Metadata: Code-Centric and Usage-Based Analysis of Known Vulnerabilities in Open-Source Software , in *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 449–460, Los Alamitos, CA, USA (2018), IEEE Computer Society.

[9] Foo, D., Yeo, J., Xiao, H. and Sharma, A.: The Dynamics of Software Composition Analysis (2019).

[10] Mir, A. M., Keshani, M. and Proksch, S.: On the Effect of Transitivity and Granularity on Vulnerability Propagation in the Maven Ecosystem , in *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 201–211, Los Alamitos, CA, USA (2023), IEEE Computer Society.

[11] Salis, V., Sotiropoulos, T., Louridas, P., Spinellis, D. and Mitropoulos, D.: PyCG: Practical Call Graph Generation in Python, in *Proceedings of the 43rd International Conference on Software Engineering*, ICSE '21, p. 1646–1657, IEEE Press (2021).

[12] Takashi, T.: Usage-Evidence-Annotated SBOM for Python, GitHub repository `https://github.com/Taka-cst/usage-evidence-annotated-sbom/tree/3a20b70`: accessed 2026-08-11.

[13] Python Packaging Authority, : pip-audit, `https://github.com/pypa/pip-audit`: accessed 2026-08-02.

[14] Open Source Vulnerabilities, : OSV Schema, `https://ossf.github.io/osv-schema/`: accessed 2026-08-02.

[15] FIRST, : Exploit Prediction Scoring System (EPSS), `https://www.first.org/epss/`: accessed 2026-08-02.

© 2026 Information Processing Society of Japan